

CARIBE TECH ARENA

PRD Técnico

BioAlert — IA Conversacional en WhatsApp

Biofood | Hackathon 24H | 2026

Premio único: \$2.000.000 COP

Pedro Noguera · CEO Biofood

■ EVALUACIÓN DE VIABILIDAD — PRE-PRD

¿Es factible en 24 horas con 4 devs?

La hipótesis original tiene 3 productos colapsados en uno: bot conversacional para padres, dashboard de alertas para cafeterías y plataforma B2B de inteligencia para proveedores. Eso son 3 semanas de trabajo. Se mata el producto B2B sin discusión — no tiene datos estructurados para soportarlo en tiempo real. El Core Value es un webhook que escucha transacciones de Biofood, las procesa con Claude API y entrega alertas + respuestas conversacionales vía WhatsApp.

Qué se recorta sin piedad:

ELIMINADO	RAZÓN
Dashboard web (Angular)	No se toca frontend en 24H
B2B para proveedores	Sin modelo de datos listo
Modelos ML propios	Sin tiempo de entrenamiento — se usa Claude
Autenticación de usuarios	El número WhatsApp ES la identidad
Históricos de años	Solo últimos 30 días para el MVP
Multi-tenant (todos los colegios)	Se fija a 1 colegio piloto con data real

Riesgos técnicos críticos:

Riesgo	Severidad	Mitigación
Meta WhatsApp API — aprobación sandbox	CRÍTICO	Pedir acceso sandbox Meta Business el DÍA 1 antes de escribir código
Latencia PostgreSQL en Lambda cold start	ALTO	Connection pooling con RDS Proxy. Sin ORM. SQL crudo.
Límites de tokens Claude API	MEDIO	Limitar contexto a últimas 20 transacciones. No más.
Número WhatsApp no vinculado en DB	MEDIO	Tabla de mapeo telefono→estudiante_id como fixture precargado
Rate limiting Meta (1000 conv/día)	BAJO	Suficiente para demo. Documentar límite.

■ VEREDICTO: Factible si el equipo no toca Angular, no discute arquitectura más de 2 horas y levanta el número de WhatsApp Business en las primeras 3 horas del hackathon.

01 · CONTEXTO & OBJETIVO CORE

Biofood opera software de control de cafeterías escolares y acumula años de data transaccional de consumos de miles de estudiantes a nivel nacional. Esa data existe en PostgreSQL y actualmente no genera ningún valor post-transacción. El costo de oportunidad es directo: los padres toman decisiones de alimentación sin información, las cafeterías reaccionan al desabasto en lugar de anticiparlo, y Biofood no tiene un diferenciador de retención frente a competidores.

El MVP resuelve un problema concreto y medible: un padre no sabe qué comió su hijo hoy, si consumió algo con un alérgeno registrado, o si su saldo se va a agotar antes del viernes. Un administrador de cafetería no sabe qué producto va a escasear mañana. Este agente convierte data transaccional existente en alertas proactivas y respuestas conversacionales en WhatsApp — sin instalar nada, sin login, sin fricción.

02 · SCOPE BOUNDARIES

■ IN SCOPE	■ OUT OF SCOPE
• Agente conversacional WhatsApp (Meta Cloud API) • Respuestas en lenguaje natural sobre consumos de estudiante vinculado al número de teléfono • Alertas proactivas: ausencia de consumo al medio día + detección de alérgeno en última transacción • Predicción de agotamiento de saldo (últimos 30 días)	• Cualquier interfaz web o cambio al frontend Angular • Modelos ML entrenados (toda inferencia va por Claude API) • Módulo B2B para proveedores externos • Flujo de registro de nuevos usuarios (fixture precargado) • Soporte multi-colegio (1 colegio piloto únicamente) • Notificaciones push, dashboards analíticos complejos

03 · USER STORIES & CRITERIOS DE ACEPTACIÓN

US-01 — Consulta conversacional padre

User Story: Como padre vinculado al sistema, quiero preguntarle al bot qué comió mi hijo hoy para tomar decisiones sobre su alimentación en casa sin llamar al colegio.

GIVEN	Soy un padre con número registrado en la tabla parent_phone_map
WHEN	Envío el mensaje '¿Qué comió Juan hoy?' al número de WhatsApp del bot
THEN	El sistema consulta transactions filtrando por student_id vinculado a mi número y fecha = hoy, construye prompt con esos datos y devuelve respuesta en lenguaje natural en < 4 segundos. Si no hay transacciones: 'Juan no ha registrado compras hoy.'

US-02 — Alerta proactiva de ausencia de consumo

User Story: Como padre, quiero recibir un mensaje automático si mi hijo no ha registrado ninguna compra antes del mediodía para verificar si está bien o si tiene problema con su saldo.

GIVEN	Son las 12:00 PM hora Colombia en un día hábil
WHEN	El Cron Job EventBridge dispara el Lambda de alertas
THEN	El sistema consulta todos los student_id activos del colegio piloto, filtra los que no tienen transacciones con created_at >= inicio del día, y envía mensaje WhatsApp al padre registrado. Proceso completo < 2 minutos para todos los estudiantes.

US-03 — Alerta crítica de alérgeno

User Story: Como padre que registró una alergia en el perfil de mi hijo, quiero recibir alerta inmediata cuando se detecte que compró un producto que contiene ese alérgeno.

GIVEN	El estudiante tiene al menos un allergen_id en student_allergens
WHEN	Se inserta nueva fila en transactions con un product_id cuya tabla product_allergens contiene ese allergen_id
THEN	El Lambda de alertas se dispara en < 30 segundos (polling cada 60 seg), y envía mensaje WhatsApp al padre con nombre del producto, alérgeno detectado y hora de compra.

US-04 — Proyección de agotamiento de saldo

User Story: Como padre, quiero saber cuándo se va a agotar el saldo de mi hijo basado en su gasto habitual para hacer la recarga a tiempo.

GIVEN	Soy un padre con número registrado
--------------	------------------------------------

WHEN	Envío el mensaje '¿Cuándo se acaba el saldo de mi hijo?'
THEN	El sistema calcula promedio de gasto diario (últimos 30 días desde transactions), divide el saldo actual (student_balance) y responde con fecha estimada de agotamiento y monto recomendado de recarga. Margen de error aceptable: ± 2 días.

US-05 — Alerta de stock crítico — admin cafetería

User Story: Como administrador de cafetería, quiero recibir alerta cuando un producto esté por debajo del umbral mínimo de stock para realizar el pedido antes de que se agote.

GIVEN	Un producto en inventory tiene <code>current_stock <= minimum_stock</code>
WHEN	El Cron Job de las 7:00 AM dispara el Lambda de revisión de inventario
THEN	El sistema envía mensaje WhatsApp al número del admin en cafeteria_admins listando los productos en stock crítico con su nivel actual. Si no hay críticos: no se envía nada.

04 · TECH STACK & ARCHITECTURE

Runtime & Infraestructura

Node.js 20 — mismo stack del backend Biofood. Cero fricción. Arquitectura serverless en AWS. Sin microservicios.

Capa	Tecnología
Canal de entrega	WhatsApp Business API (Meta Cloud API)
Motor IA conversacional	Claude API — claude-sonnet-4-20250514
Backend / Lambdas	Node.js 20 (Express handler)
Base de datos	PostgreSQL — Biofood Global DB (existente)
Connection pooling	RDS Proxy (sin ORM, SQL crudo)
Predicción	Lógica en Lambda (promedio móvil simple)
Alertas programadas	EventBridge Cron → Lambda
Webhook entrante	API Gateway (HTTP API) → Lambda
Secrets	SSM Parameter Store
Sesiones conversacionales	DynamoDB (TTL 1 hora)

Flujo de datos

1. Transacción ocurre en cafetería Biofood 2. API REST registra en PostgreSQL 3. Polling Lambda cada 60 seg detecta nueva fila 4. Motor evalúa: ¿Alérgeno? → Alerta inmediata al padre (<30 seg) ¿Stock bajo? → Alerta a cafetería (7AM cron) ¿Sin consumo? → Alerta a padre (12PM cron) 5. Datos alimentan cálculo de proyección de saldo 6. WhatsApp Cloud API entrega mensajes a usuarios

05 · MODELO DE DATOS

Tablas existentes en Biofood (confirmar nombres reales con el equipo):

```
students (id, name, grade, school_id, balance) transactions (id, student_id, product_id, amount,
created_at) products (id, name, category, price)
```

Tablas nuevas — fixtures precargados ANTES del hackathon:

```
student_allergens (student_id, allergen_name) product_allergens (product_id, allergen_name)
parent_phone_map (phone_e164, student_id) cafeteria_admins (phone_e164, school_id) inventory
(product_id, school_id, current_stock, minimum_stock) conversations (phone_e164, session_json,
updated_at) -- DynamoDB TTL
```

06 · DATASET REQUIREMENTS

El equipo debe recibir los siguientes mocks en la DB de pruebas antes de las primeras 2 horas del hackathon. Sin esto no hay demo.

parent_phone_map — mínimo 10 registros

```
{ "phone_e164": "+573001234567", "student_id": "uuid-del-estudiante" }
```

student_allergens — mínimo 3 estudiantes con alergias

```
{ "student_id": "uuid-del-estudiante", "allergen_name": "mani" }
```

product_allergens — mínimo 5 productos mapeados

```
{ "product_id": "uuid-del-producto", "allergen_name": "mani" }
```

inventory — todos los productos del colegio piloto

```
{ "product_id": "uuid-del-producto", "school_id": "uuid-del-colegio", "current_stock": 12,  
  "minimum_stock": 20 }
```

transactions — mínimo 30 días de historia para los 10 estudiantes mock

```
-- BLOQUEADOR CRITICO: sin este dataset el modelo de proyeccion de saldo -- no tiene con que  
calcular. Es el primer entregable del dia 1.
```


07 · MVP CHECKLIST & MÉTRICAS DE ÉXITO

Alcance del hackathon:

- Integración WhatsApp Business API (Meta Cloud API)
- Motor conversacional con Claude API (claude-sonnet-4-20250514)
- Alerta de alérgenos en tiempo real (< 30 segundos)
- Alerta de ausencia de consumo diaria (Cron 12PM)
- Consulta conversacional: '¿Qué comió mi hijo hoy?'
- Proyección de agotamiento de saldo por promedio móvil
- Alerta de stock crítico para admin cafetería (Cron 7AM)
- Conexión a API REST Biofood + PostgreSQL existente

Métricas de éxito:

Métrica	Meta
Tiempo de respuesta conversacional	< 4 segundos
Detección de alérgenos	100% (regla determinista, no ML)
Entrega de alerta crítica desde transacción	< 30 segundos
Proceso de alerta masiva (todos los estudiantes)	< 2 minutos
Precisión proyección de saldo	Margen de error ±2 días

Cronograma de 24 horas:

Hora	Entregable
H0 – H3	Setup WhatsApp Business API sandbox + número activo + fixture DB cargado
H3 – H8	Lambda ConversationHandler: webhook → Claude API → respuesta WhatsApp
H8 – H13	Motor de alertas: polling alérgenos + Cron 12PM ausencia de consumo
H13 – H18	Cron 7AM stock crítico + proyección de saldo conversacional
H18 – H22	Integración completa + pruebas end-to-end con data real
H22 – H24	Demo prep + casos de uso listos para presentación

Primer bloqueo a resolver antes de escribir código: Número de WhatsApp Business habilitado con Meta Cloud API en modo sandbox. Sin esto no hay demo. Asignar a 1 dev exclusivamente durante las primeras 3 horas.